

Dépôt Git & Versioning — FutureKawa

Conventions, branches et workflow Git

Projet : FutureKawa

Version : 1.0

Date : 02/07/2026

Équipe : Rudy · Florian · Noé · Josselin

Sommaire

Documentation Git — FutureKawa

01 — Pourquoi GitLab

02 — Convention de branches

03 — Convention de commits

Documentation Git — FutureKawa

Projet	FutureKawa
Document	Conventions Git et workflow de versioning
Version	1.0
Date	02/07/2026
Équipe	Rudy · Florian · Noé · Josselin

Vue d'ensemble

Ce document décrit les conventions Git adoptées pour le projet FutureKawa : choix de plateforme, stratégie de branches et format des commits.

Navigation

#	Document	Contenu
1	Choix de GitLab	Pourquoi GitLab, fonctionnalités utilisées, organisation du monorepo
2	Convention de branches	Git Flow simplifié, règles de nommage, cycle de vie
3	Convention de commits	Format Conventional Commits, types, scopes, exemples

01 — Pourquoi GitLab

Contexte

FutureKawa est un projet académique développé en équipe de 4 personnes. Le choix de la plateforme Git a été fait en début de projet pour toute la durée du développement.

Pourquoi GitLab plutôt que GitHub

Critère	GitLab	GitHub
CI/CD intégré	Natif, sans plugin tiers	Actions (externe)
Registry Docker	Inclus dans le projet	Packages séparé
Runners Docker	Disponibles gratuitement	Limités sur Free
SAST / Secret Detection	Inclus sur tier Free	Payant (Advanced Security)
Merge Requests	Revue de code intégrée	Pull Requests équivalentes
Auto DevOps	Disponible	Non natif

GitLab a été retenu principalement parce que le **CI/CD, le registry Docker et les scanners de sécurité** (SAST, détection de secrets) sont disponibles sans abonnement. Cela permet de maintenir une pipeline complète sans coût supplémentaire.

Organisation du dépôt

FutureKawa utilise un **monorepo** : tous les services coexistent dans un seul dépôt. Ce choix facilite la coordination entre services et permet de déclencher les tests CI uniquement sur les fichiers modifiés (via les règles `changes:` du pipeline).

```

futurekawa/                                ← dépôt unique GitLab
├─ API-siege/
├─ API-exploitation/
├─ front-siege/
├─ front-exploitation/
├─ simu-sensor/
├─ broker/
├─ iot/
├─ infra/
└─ .gitlab-ci.yml                          ← pipeline unique pour tous les
services

```

Fonctionnalités GitLab utilisées

Fonctionnalité	Usage dans FutureKawa
CI/CD Pipelines	6 stages : lint → test → build → staging → security → deploy
Registry Docker	Stockage des images buildées (<code>registry.gitlab.com/r_dy/futurekawa/</code>)
Merge Requests	Revue de code obligatoire avant merge sur <code>dev</code> ou <code>main</code>
SAST (Semgrep)	Analyse statique de sécurité Python + JavaScript
Secret Detection	Détection de secrets dans l'historique Git
Branch Protection	<code>dev</code> et <code>main</code> protégées — merge via MR uniquement
Job artifacts	Rapports JUnit et couverture Cobertura accessibles dans l'interface

Accès au dépôt

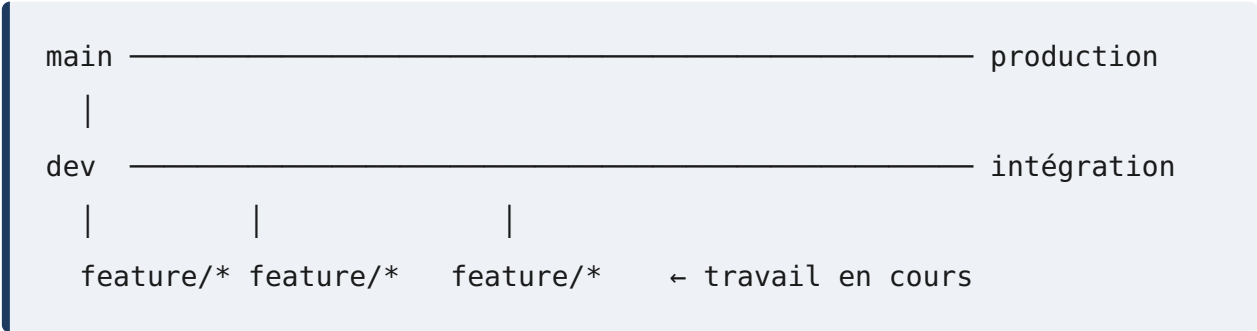
```
https://gitlab.com/R_dy/futurekawa
```

La branche de travail principale est `dev` . La branche `main` reçoit uniquement les versions stables validées par la pipeline complète.

02 — Convention de branches

Git Flow simplifié

Le projet utilise un **Git Flow à deux branches permanentes** :



Branches permanentes

Branche	Rôle	Protection
main	Code de production — déclenche le deploy CI	Protégée — merge via MR uniquement
dev	Intégration continue — déclenche build + staging	Protégée — merge via MR uniquement

Branches temporaires

Préfixe	Usage	Exemple
feature/	Nouvelle fonctionnalité	feature/invitation-token
fix/	Correction de bug	fix/alert-duplicate
chore/	Tâche technique sans impact fonctionnel	chore/cleanup-migrations
docs/	Documentation uniquement	docs/bdd-exploitation

Cycle de vie d'une branche

1. Partir de dev
`git checkout dev && git pull`
`git checkout -b feature/ma-fonctionnalite`
2. Développer + commiter
`git add <fichiers>`
`git commit -m "feat(scope): description"`
3. Merger dans dev via Merge Request GitLab
→ review → CI verte → merge
4. Sur dev : build + staging automatiques en CI
→ smoke test OK → merge dans main
5. Sur main : deploy CI (terraform validate + gate manuel)

Règles importantes

- **Jamais commiter directement sur `dev` ou `main`** — toujours passer par une branche `feature/`.
- Une branche = une fonctionnalité ou un fix. Ne pas mélanger plusieurs sujets.
- Supprimer la branche après merge (`git branch -d feature/ma-
branche`).
- Toujours rebaser sur `dev` avant d'ouvrir une Merge Request si la branche a pris du retard.

Nommage des branches

```
feature/<sujet-en-kebab-case>
fix/<composant>-<probleme>
chore/<description-courte>
docs/<sujet>
```

Exemples valides :


```
feature/invitation-token  
feature/exploitation-incidents  
fix/api-siege-auth-cookie  
chore/remove-obsolete-migrations  
docs/livrables-documentation
```

03 — Convention de commits

Format Conventional Commits

Tous les commits suivent la spécification **Conventional Commits** :

```
<type>(<scope>): <description courte>
```

```
[corps optionnel]
```

Types disponibles

Type	Usage
feat	Nouvelle fonctionnalité
fix	Correction de bug
chore	Tâche technique (dépendances, config, cleanup)
docs	Documentation uniquement
test	Ajout ou modification de tests
refactor	Refactorisation sans changement de comportement
style	Formatage, espaces, lint (sans impact sur la logique)
ci	Modification du pipeline CI/CD

Scopes utilisés dans le projet

Scope	Concerne
siege	API Siège
exploitation	API Exploitation
front-siege	Frontend Siège

Scope	Concerne
front-exploitation	Frontend Exploitation
simu-sensor	Simulateur capteur
broker	Broker MQTT
iot	Code ESP32
infra	Infrastructure Terraform
ci	Pipeline GitLab CI
schema	Schémas BDD PlantUML
seed	Données de démonstration
cleanup	Nettoyage de code ou fichiers

Exemples de commits

```

feat(exploitation): ajout endpoint /api/incidents/ avec filtres par
entrepôt
fix(siege): corrige la validation du token d'invitation expiré
test(api-siege): couverture des endpoints d'authentification
chore(cleanup): supprime les migrations obsolètes de l'API
exploitation
docs(bdd): met à jour le schéma PlantUML avec la table INCIDENT
ci: ajoute le job smoke_test sur la stack Quito
refactor(front-exploitation): extrait LotCard en composant autonome

```

Règles

- La description est en **français**, courte, à l'impératif.
- Le scope est **optionnel** mais recommandé dès qu'une modification touche un service précis.
- Pas de point final en fin de description.
- Le corps (optionnel) explique le **pourquoi**, pas le quoi.